

# 1. TME 3 — Perceptron et SVM

## 1.1. Introduction

L'objectif de ce TME est d'étudier plusieurs méthodes de classification binaire : le perceptron, ses variantes d'apprentissage par descente de gradient, les projections non linéaires, la perte hinge pénalisée, puis les SVM. Nous utilisons d'abord des données artificielles en deux dimensions, puis les données USPS de chiffres manuscrits.

Dans tout le rapport, les labels sont encodés en  $-1$  et  $+1$ . Le classifieur linéaire prédit donc le signe de  $x^T w$ .

## 1.2. Perceptron

La perte perceptron utilisée est

$$\ell(w, x, y) = \max(0, -yx^T w).$$

Cette perte est nulle lorsque l'exemple est bien classé avec une marge positive, et strictement positive lorsqu'il est mal classé. Le gradient utilisé dans le code correspond à la moyenne des contributions des exemples mal classés.

Le modèle est implémenté dans une classe `Lineaire`. Cette classe permet de changer la fonction de coût, le gradient, le nombre d'itérations, le pas de gradient, la projection utilisée, ainsi que le type d'apprentissage : batch complet, stochastique ou mini-batch.

## 1.3. Données USPS : classification 6 contre 9

Nous isolons d'abord deux classes, les chiffres 6 et 9. Quelques exemples des images utilisées sont représentés ci-dessous.



Figure 1: Exemples USPS pour les classes 6 et 9.

Le perceptron est ensuite entraîné sur ces deux classes. Les résultats obtenus sont les suivants.

| Expérience  | Score train | Score test | Erreur test |
|-------------|-------------|------------|-------------|
| USPS 6 vs 9 | 0.998       | 0.991      | 0.009       |

Le coût final est très faible, environ 0.0001. Le modèle distingue donc très bien les deux chiffres.

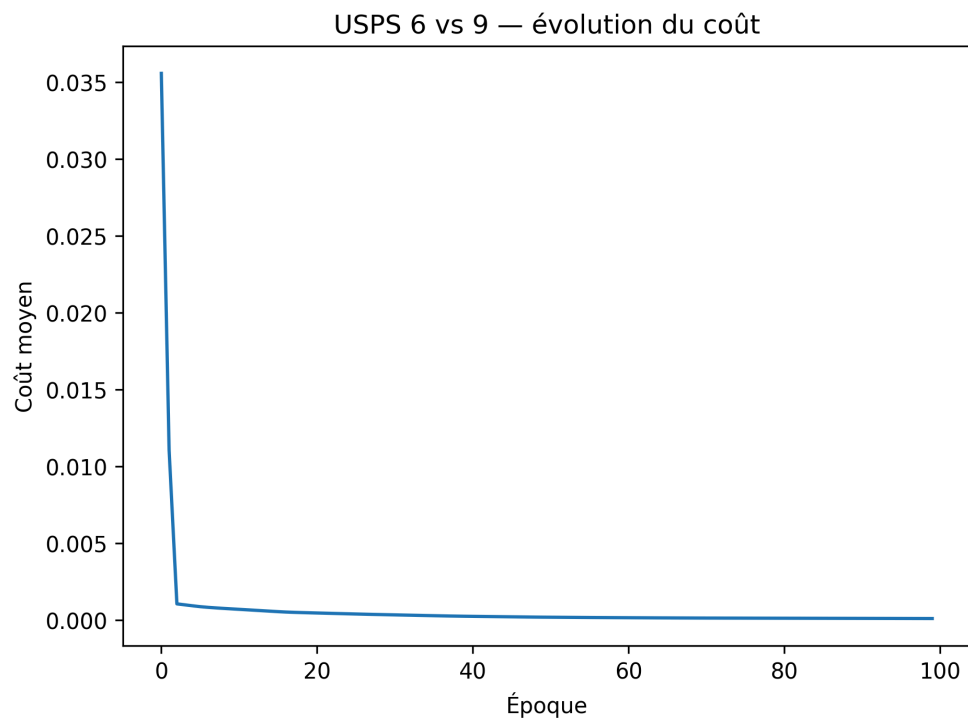


Figure 2: Évolution du coût du perceptron sur USPS 6 vs 9.

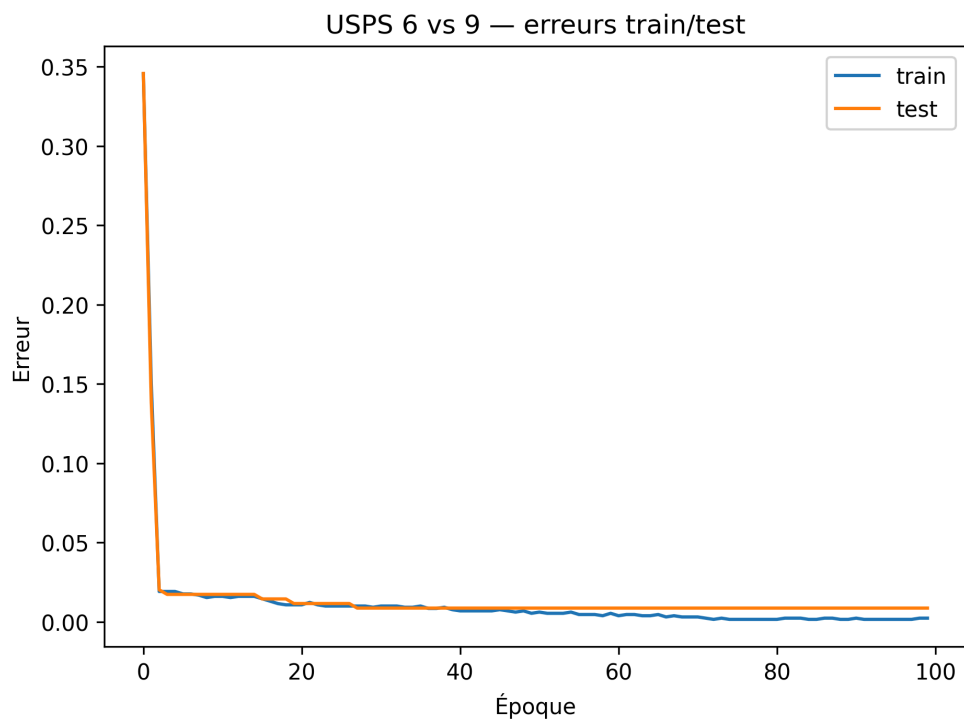


Figure 3: Erreurs train et test pour USPS 6 vs 9.

L'erreur de test reste très proche de l'erreur d'apprentissage. On ne constate donc pas de sur-apprentissage marqué.

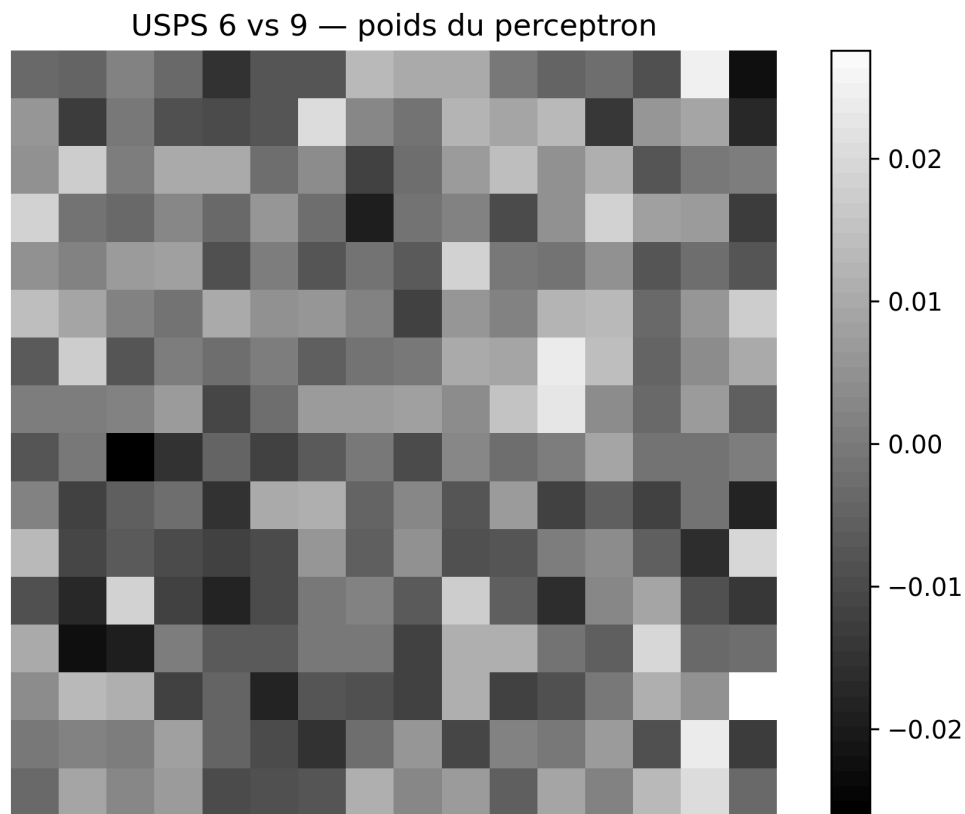


Figure 4: Matrice de poids du perceptron pour USPS 6 vs 9.

Cette matrice s'interprète comme un masque discriminant. Les pixels de poids positif favorisent la classe positive, ici le 9, tandis que les pixels de poids négatif favorisent la classe négative, ici le 6.

#### 1.4. Classification 6 contre toutes les autres classes

On entraîne ensuite un perceptron pour séparer le chiffre 6 de tous les autres chiffres. Le problème est plus difficile, car la classe négative est très hétérogène.

| Expérience    | Score train | Score test | Erreur test |
|---------------|-------------|------------|-------------|
| USPS 6 vs all | 0.980       | 0.969      | 0.031       |

Le score reste élevé, mais l'erreur test est plus importante que dans le cas 6 contre 9. Cela confirme que le problème est plus difficile.

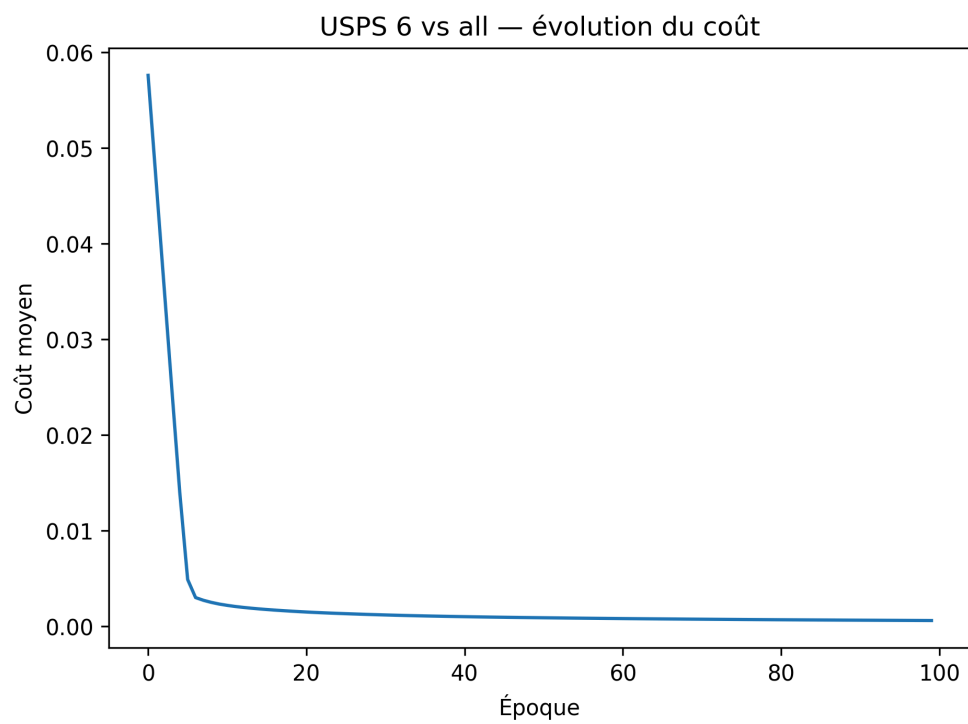


Figure 5: Évolution du coût pour USPS 6 vs all.

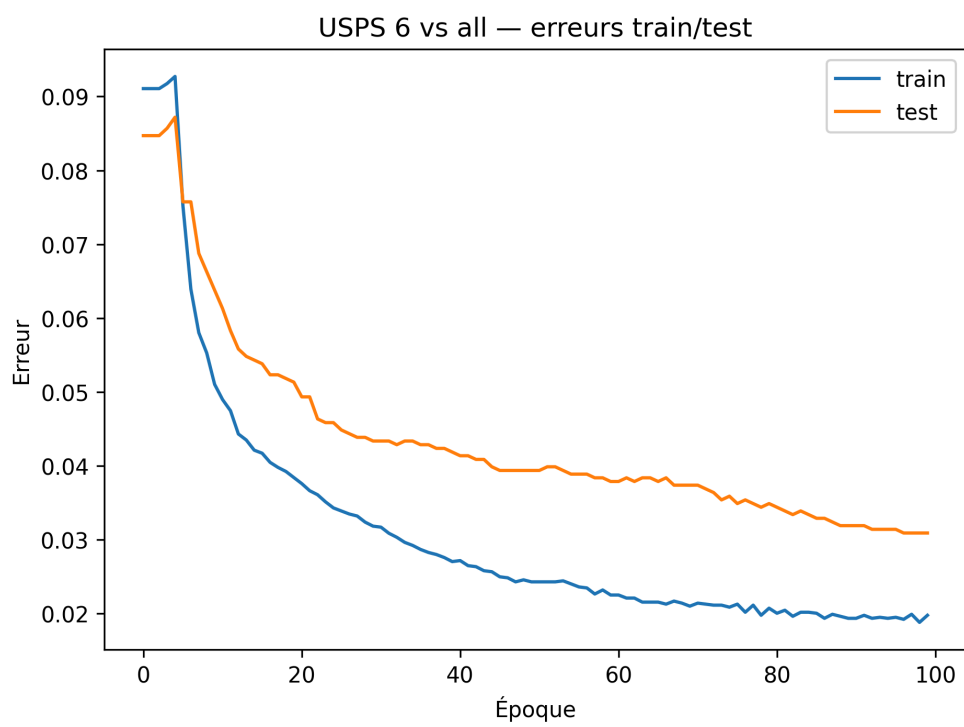


Figure 6: Erreurs train et test pour USPS 6 vs all.

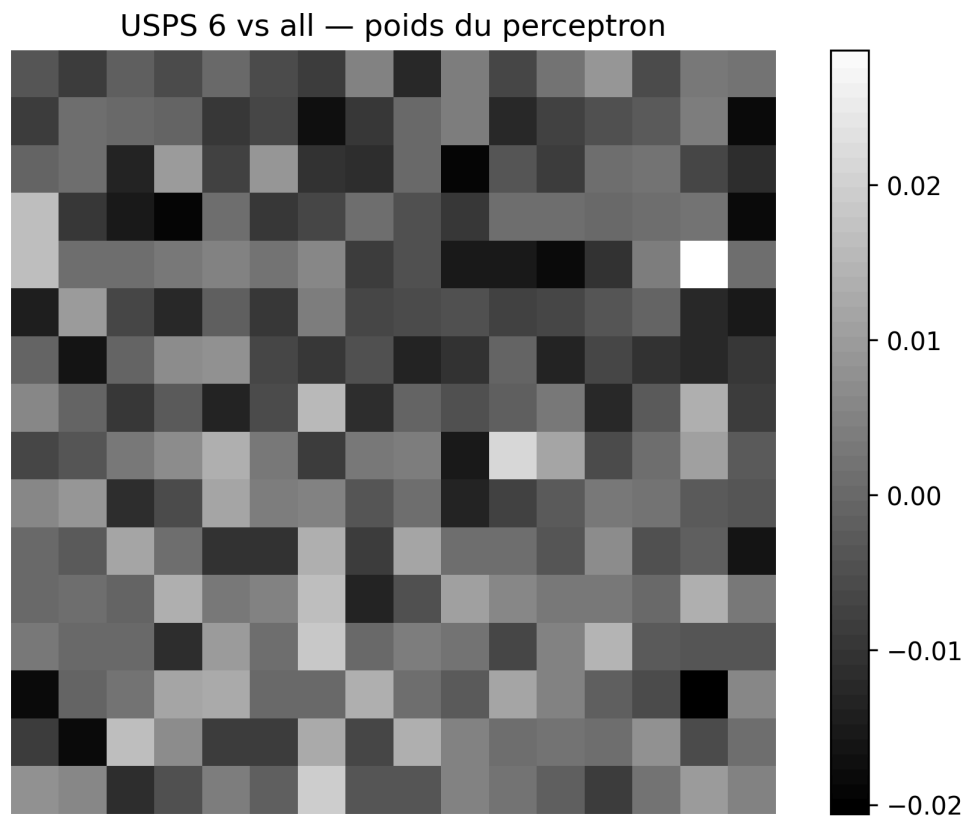


Figure 7: Matrice de poids pour USPS 6 vs all.

La matrice de poids est moins facile à interpréter que dans le cas 6 contre 9, car la classe négative regroupe plusieurs chiffres différents.

### 1.5. Descente batch, stochastique et mini-batch

Nous comparons ensuite trois variantes d'apprentissage.

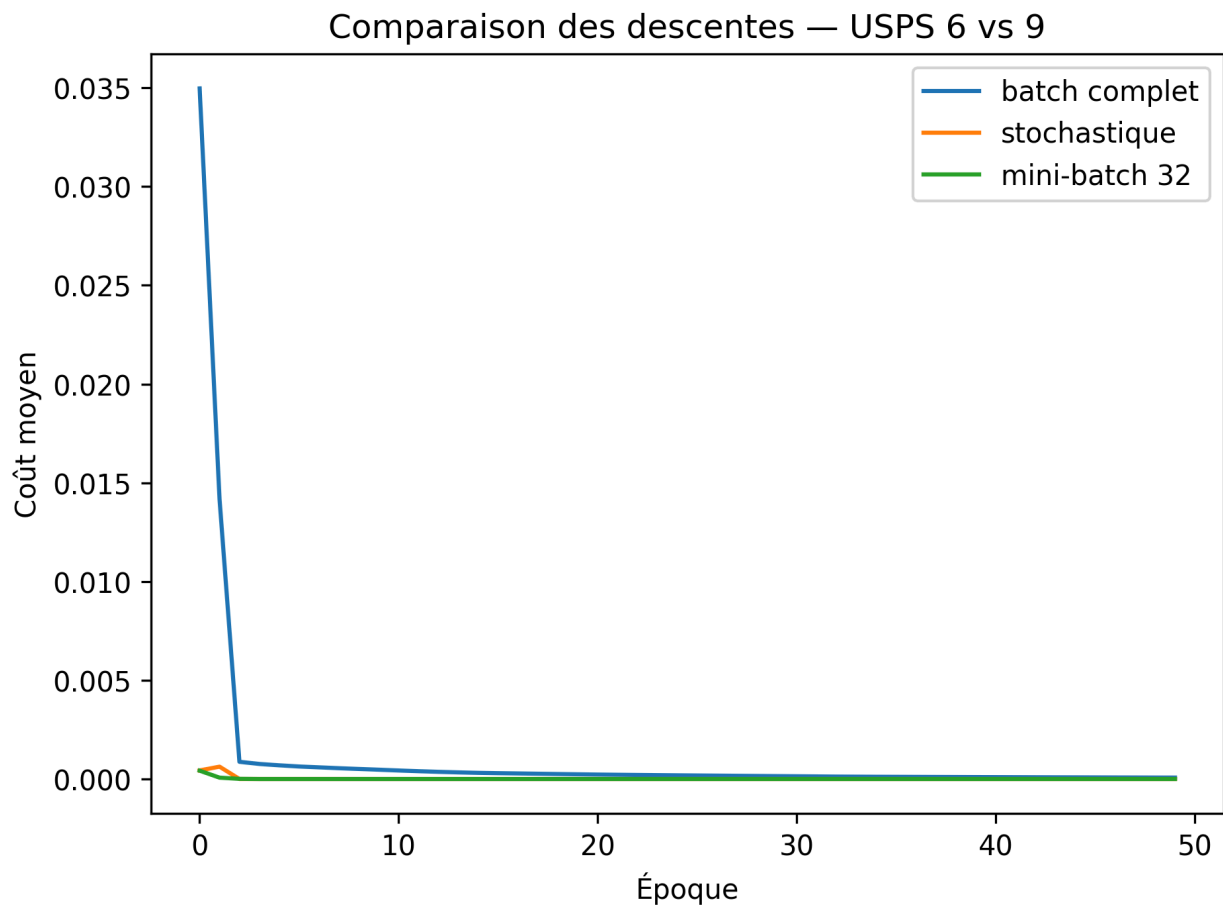


Figure 8: Comparaison des descentes batch, stochastique et mini-batch.

| Méthode       | Coût final | Score train | Score test |
|---------------|------------|-------------|------------|
| Batch complet | 0.0001     | 0.995       | 0.988      |
| Stochastique  | 0.0000     | 1.000       | 0.997      |
| Mini-batch 32 | 0.0000     | 1.000       | 1.000      |

Les trois méthodes convergent bien sur ce problème. Le mini-batch donne ici le meilleur score test. La descente stochastique et la descente mini-batch progressent vite, car elles effectuent plus de mises à jour pendant une époque.

## 1.6. Projection polynomiale

Un modèle linéaire dans l'espace initial ne peut produire qu'une frontière linéaire. Pour augmenter son expressivité, on projette les données dans un espace de plus grande dimension.

$$1, x_1, x_2, \dots, x_d, x_1^2, x_1x_2, \dots, x_d^2.$$

| Jeu de données     | Score train | Erreur train |
|--------------------|-------------|--------------|
| Deux gaussiennes   | 1.000       | 0.000        |
| Quatre gaussiennes | 0.998       | 0.002        |
| Échiquier          | 0.505       | 0.495        |

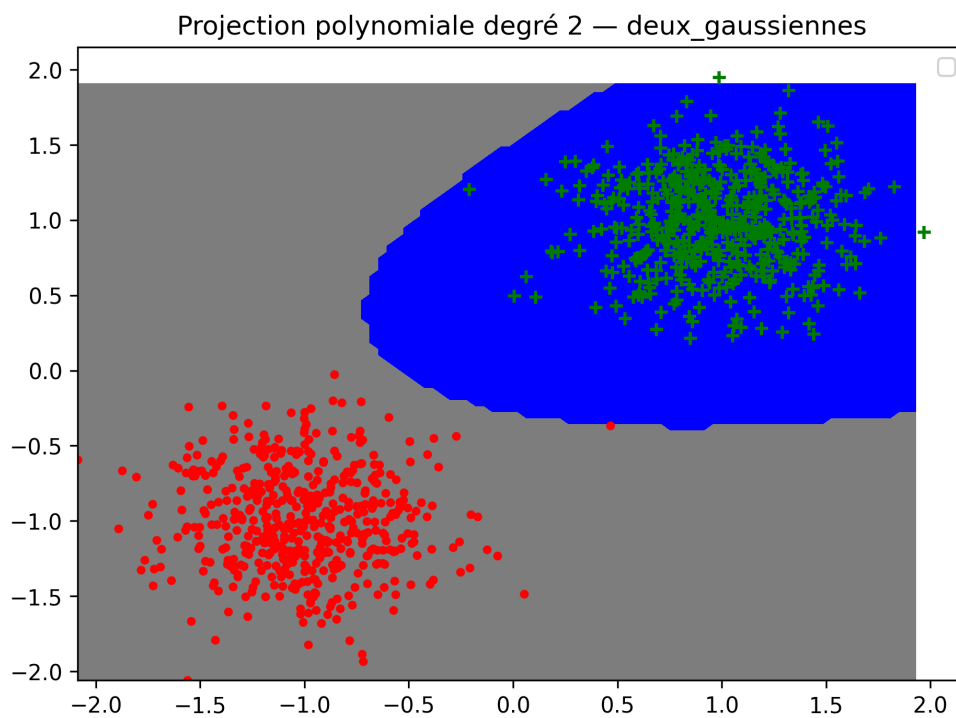


Figure 9: Projection polynomiale sur deux gaussiennes.

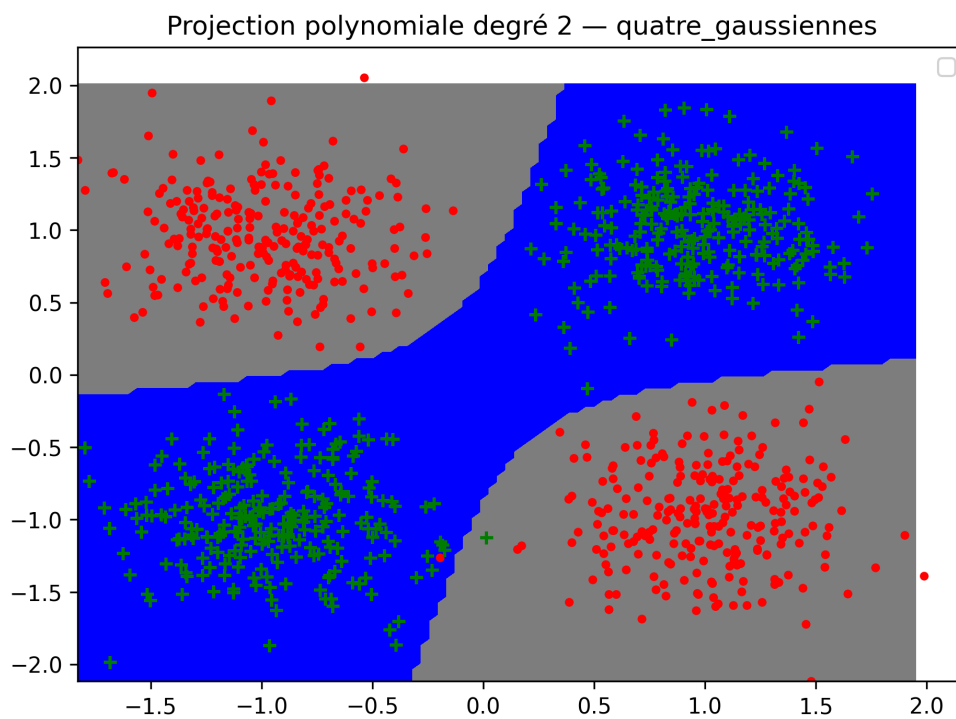


Figure 10: Projection polynomiale sur quatre gaussiennes.

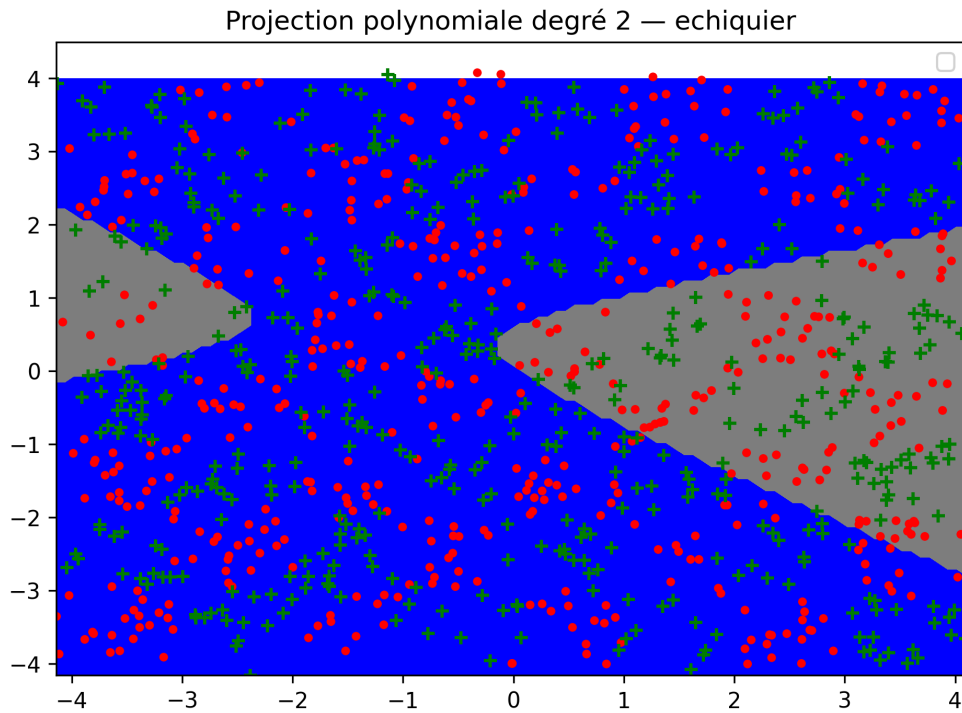


Figure 11: Projection polynomiale sur l'échiquier.

La projection polynomiale fonctionne très bien pour les deux gaussiennes et les quatre gaussiennes. En revanche, elle échoue sur l'échiquier : le score est proche de 0.5, donc proche du hasard. Une frontière quadratique reste insuffisante pour cette structure.

### 1.7. Projection gaussienne

Nous utilisons ensuite une projection gaussienne sur des points de base  $b_1, \dots, b_m$  :

$$\varphi(x) = \left( \exp\left(-\|x - b_1\|_{\frac{1}{2\sigma^2}}^2\right), \dots, \exp\left(-\|x - b_m\|_{\frac{1}{2\sigma^2}}^2\right) \right).$$

| Nombre de points de base | $\sigma$ | Score train | Erreur train |
|--------------------------|----------|-------------|--------------|
| 20                       | 0.2      | 0.525       | 0.475        |
| 20                       | 1.0      | 0.533       | 0.467        |
| 80                       | 0.2      | 0.619       | 0.381        |
| 80                       | 1.0      | 0.584       | 0.416        |



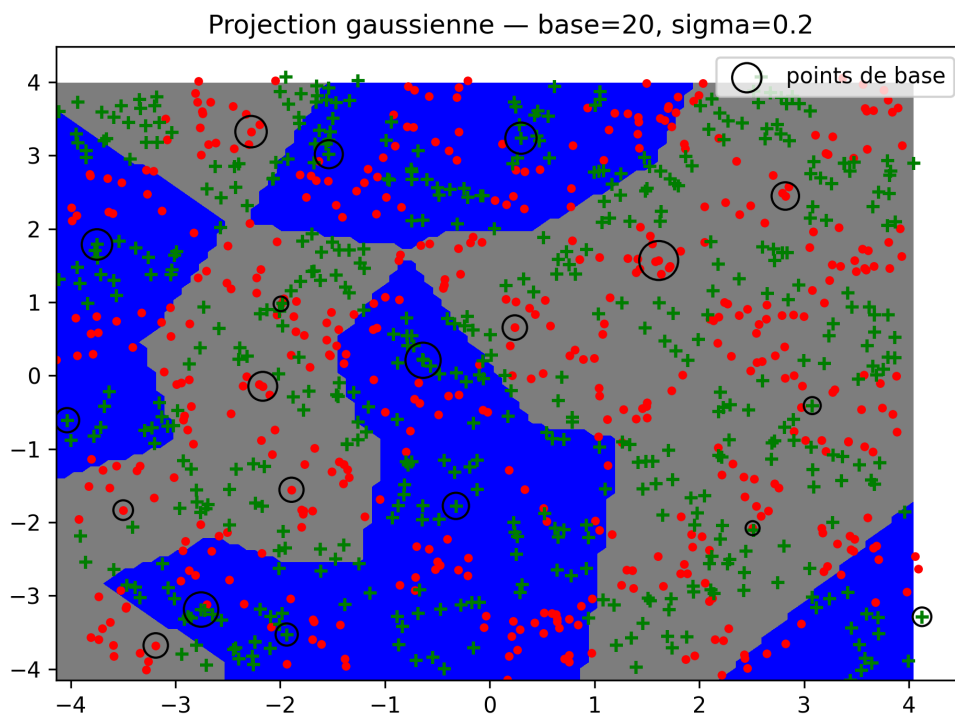


Figure 12: Projection gaussienne avec peu de points de base et petit sigma.

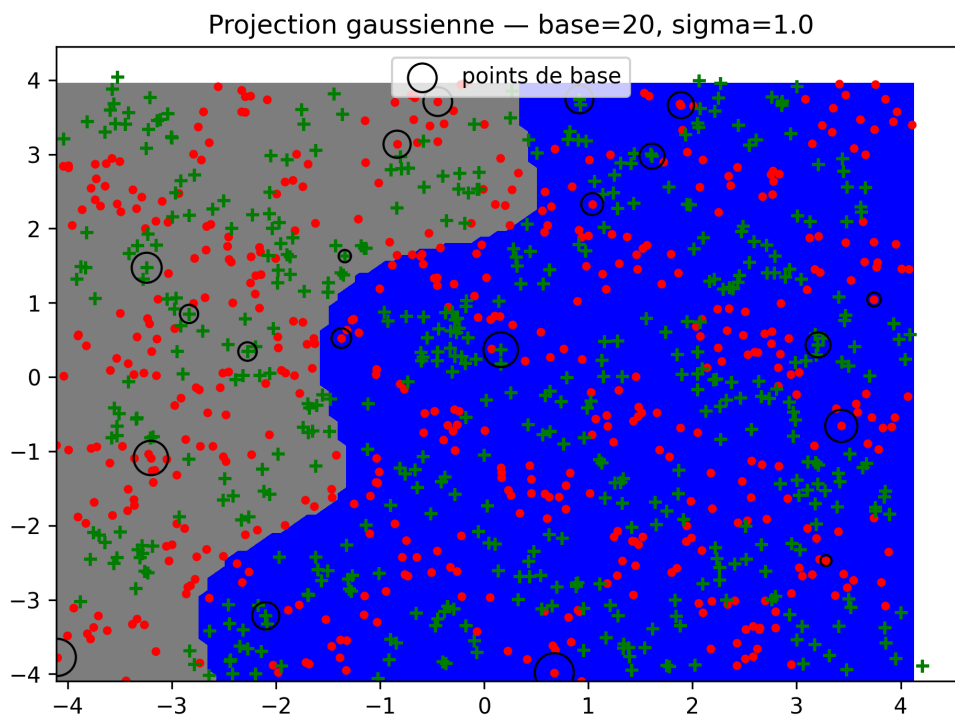


Figure 13: Projection gaussienne avec peu de points de base et grand sigma.

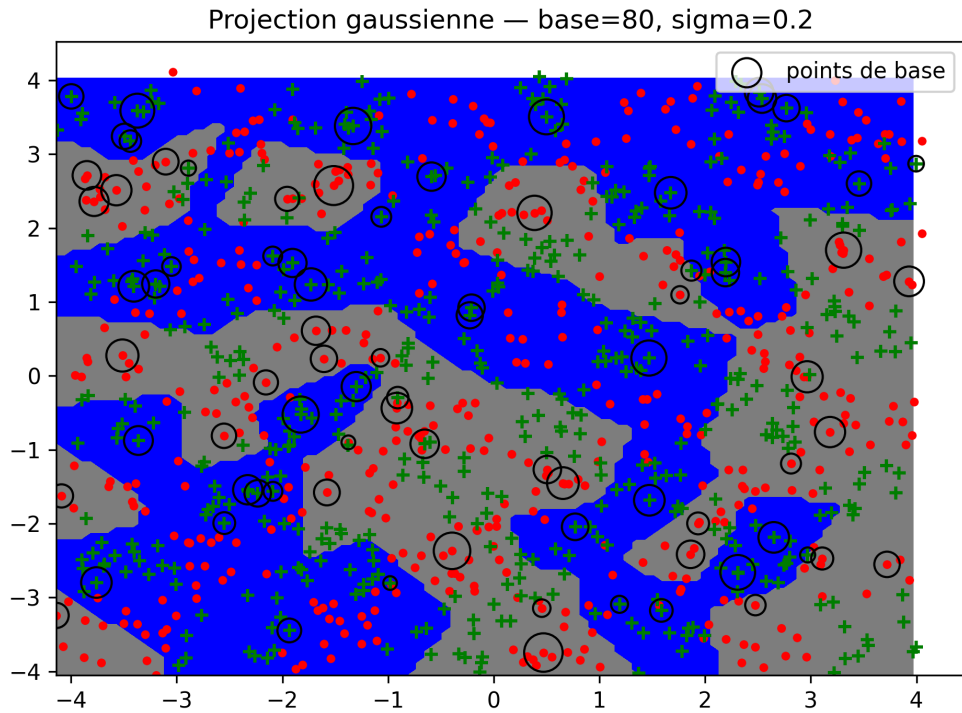


Figure 14: Projection gaussienne avec beaucoup de points de base et petit sigma.

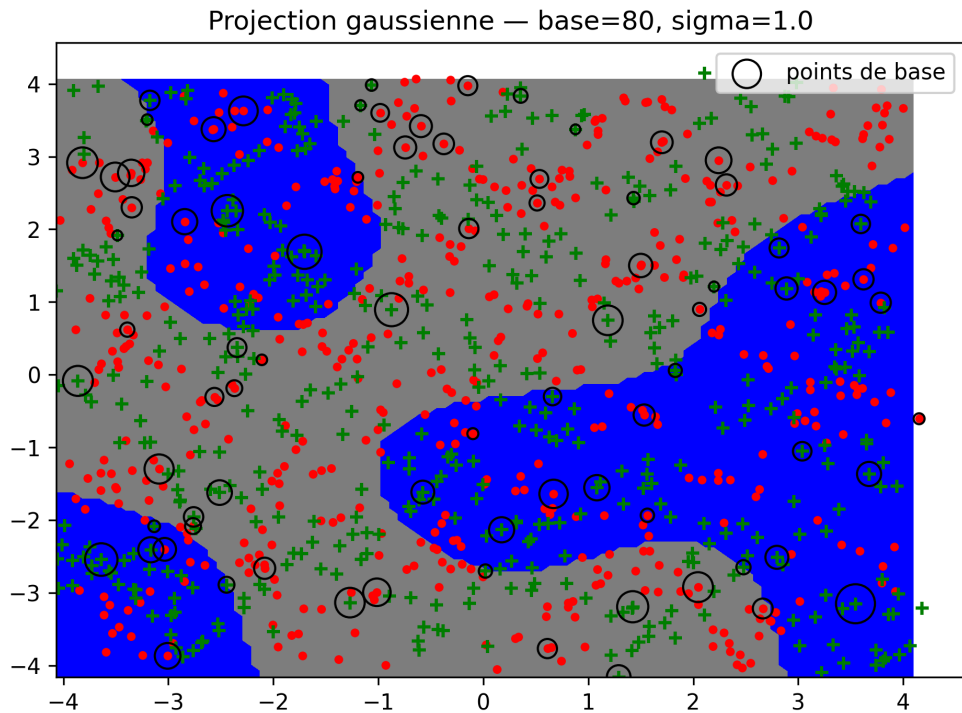


Figure 15: Projection gaussienne avec beaucoup de points de base et grand sigma.

La meilleure configuration testée ici est  $\text{nb\_base} = 80$  et  $\sigma = 0.2$ , avec un score train de 0.619. L'augmentation du nombre de points de base améliore l'expressivité du modèle. Les performances restent cependant limitées sur l'échiquier.

### 1.8. Perte hinge et pénalisation

On remplace ensuite la perte perceptron par une perte hinge pénalisée :

$$\ell(w, x, y) = \max(0, \alpha - yx^T w) + \lambda \|w\|^2.$$

| $\alpha$ | $\lambda$ | Score train | Coût final |
|----------|-----------|-------------|------------|
| 0.5      | 1e-4      | 0.644       | 0.3722     |
| 1.0      | 1e-4      | 0.635       | 0.7857     |
| 2.0      | 1e-4      | 0.617       | 1.6716     |
| 1.0      | 1e-2      | 0.599       | 0.9602     |
| 1.0      | 1e-1      | 0.598       | 0.9960     |

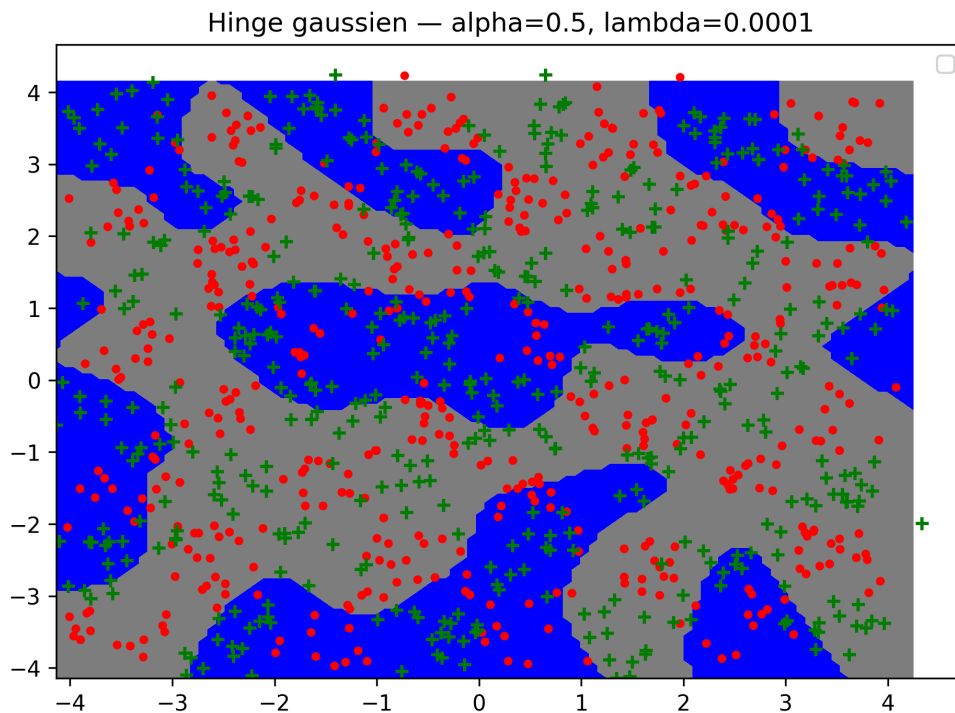


Figure 16: Perte hinge avec  $\alpha=0.5$  et  $\lambda=1e-4$ .

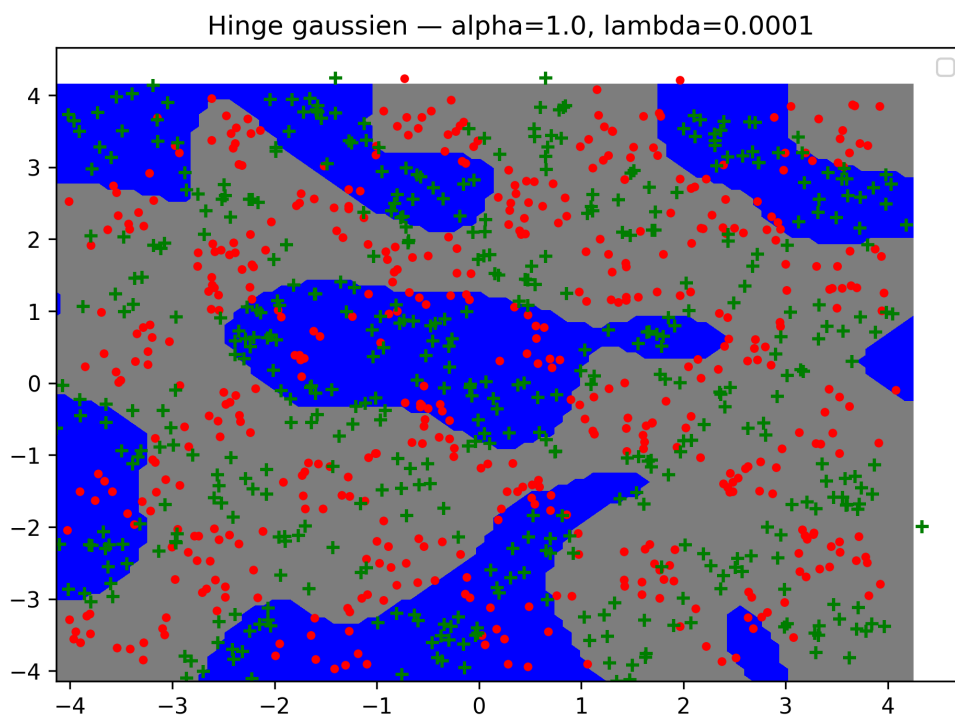


Figure 17: Perte hinge avec  $\alpha=1.0$  et  $\lambda=1e-4$ .

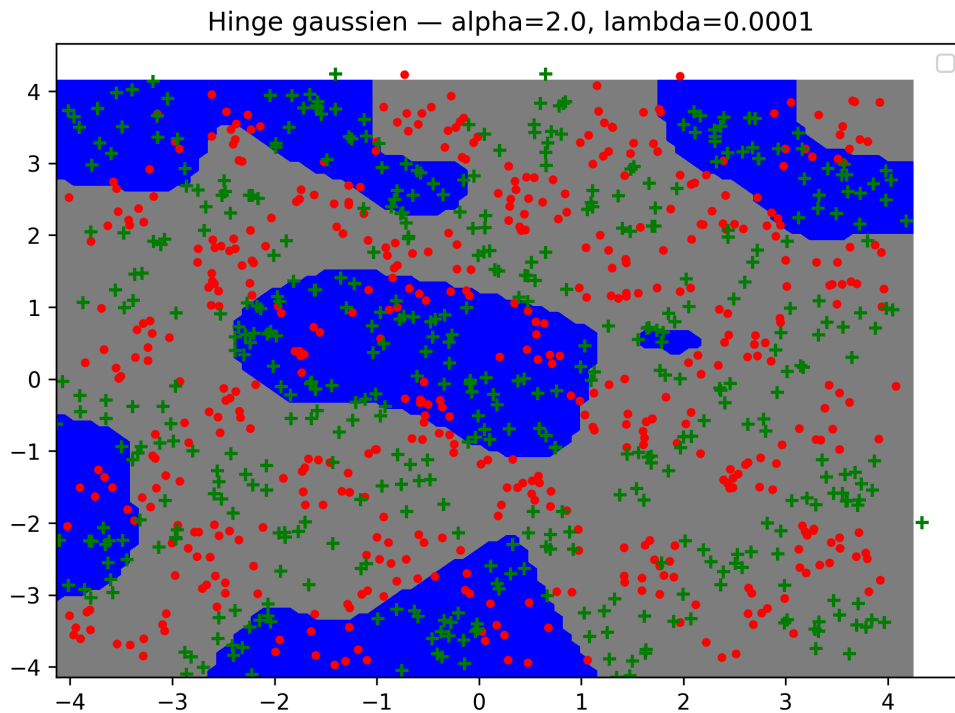


Figure 18: Perte hinge avec  $\alpha=2.0$  et  $\lambda=1e-4$ .

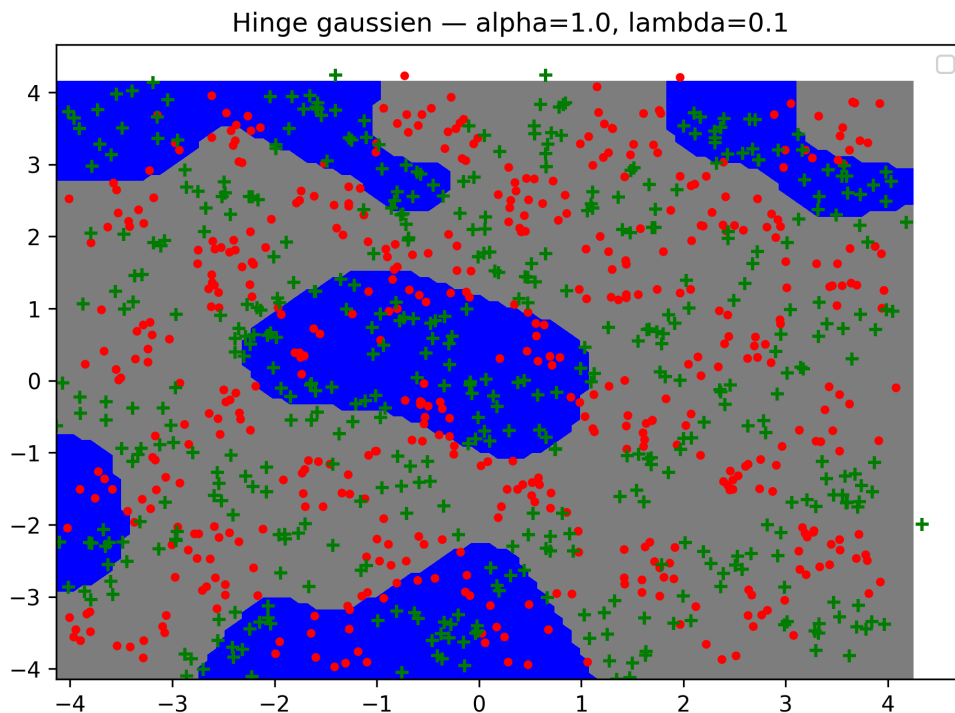


Figure 19: Perte hinge avec  $\alpha=1.0$  et  $\lambda=1e-1$ .

Lorsque  $\alpha$  augmente, le coût final augmente car la marge demandée est plus grande. Lorsque  $\lambda$  augmente, les poids sont davantage pénalisés, ce qui peut entraîner du sous-apprentissage. Ici, le meilleur score parmi les configurations testées est obtenu avec  $\alpha = 0.5$  et  $\lambda = 10^{-4}$ .

### 1.9. SVM et Grid Search

Enfin, nous utilisons les SVM de `sklearn`. Les paramètres sont choisis par validation croisée avec `GridSearchCV`.

| Données   | Meilleur noyau | Meilleurs paramètres | Score test | Nb vecteurs supports |
|-----------|----------------|----------------------|------------|----------------------|
| Échiquier | RBF            | $C=1$ , $\gamma=10$  | 0.807      | 550                  |

|             |     |                 |       |     |
|-------------|-----|-----------------|-------|-----|
| USPS 6 vs 9 | RBF | C=1, gamma=0.01 | 0.997 | 150 |
|-------------|-----|-----------------|-------|-----|

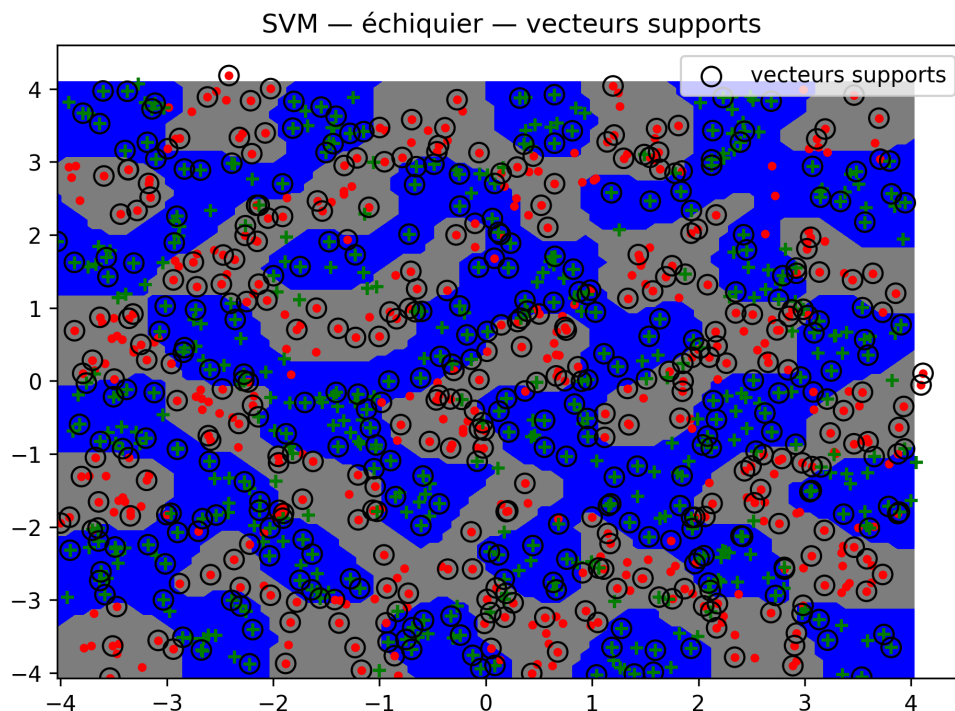


Figure 20: SVM sur l'échiquier avec vecteurs supports.

Sur l'échiquier, le SVM RBF obtient un score test de 0.807, nettement supérieur aux projections précédentes. Sur USPS 6 contre 9, le score test atteint 0.997, ce qui est légèrement meilleur que le perceptron simple.

Les vecteurs supports sont les exemples qui participent directement à la définition de la frontière. Sur l'échiquier, leur nombre est élevé, ce qui reflète la complexité de la frontière.

## 1.10. Conclusion

Ce TME met en évidence les limites et les extensions naturelles des modèles linéaires. Le perceptron fonctionne bien lorsque les données sont presque linéairement séparables, comme pour USPS 6 contre 9. Le cas 6 contre toutes les autres classes est plus difficile, mais reste bien traité.

Les projections polynomiales permettent de résoudre certains problèmes non linéaires simples, comme les quatre gaussiennes, mais échouent sur l'échiquier. Les projections gaussiennes améliorent légèrement les résultats, mais restent limitées avec les paramètres testés. La perte hinge introduit une notion de marge et rapproche le modèle des SVM.

Enfin, les SVM avec noyau RBF donnent les meilleurs résultats sur les données complexes, notamment l'échiquier et USPS 6 contre 9.